



DataScientest

Rapport Final - Prédiction de la Satisfaction Client dans la Supply Chain: Exemple d'étude : Les Banques en Ligne en France

Groupe : Mathieu Karr - Catherine Otieno Strandberg - Ikram El Hasnaoui

Mai 24 cds

Tuteur : Sébastien Sime

Date du rendu : 02 Mars 2024

Sommaire

Introduction	2
Collecte et Exploration des Données	
Web Scraping avec BeautifulSoup.....	3
Exploration et Pre-Processing.....	4
Visualizations.....	5
Modélisation et Prédiction	
Vectorisation des avis.....	13
Clustering et modélisation des thématiques.....	15
Sélection de modèles.....	17
Optimisation de modèles.....	19
Interprétation des résultats.....	21
Bert Topics	21
Conclusion.....	22
Annexes.....	25
References.....	26

Introduction

Dans le cadre de notre formation en Data Science à DataScientest, nous avons réalisé un projet Fil Rouge visant à analyser et prédire la satisfaction client dans la supply chain, en prenant l'exemple des banques en ligne.

La satisfaction client est un enjeu stratégique majeur pour les entreprises, car elle influence directement la fidélisation, l'image de marque et la rentabilité. Comprendre les facteurs déterminants de la satisfaction et de l'insatisfaction permet aux entreprises d'adopter des actions correctives et d'optimiser leurs services, un atout concurrentiel décisif pour leur compétitivité et leur survie. En 2024, le marché des solutions de Voice of the Customer (VoC) et de gestion de l'expérience client a atteint 19 milliards d'euros dans le monde d'après Mordor Intelligence, et une croissance de 17% est prévue d'ici 2026.

L'essor du digital a conduit à une explosion des avis en ligne sur des plateformes spécialisées comme TrustPilot. Ces avis sont une mine d'informations précieuses pour les entreprises, mais leur volume et leur hétérogénéité rendent leur analyse complexe. Grâce aux avancées en traitement du langage naturel (NLP), il est désormais possible d'exploiter ces données de manière automatisée pour en extraire des tendances et des thématiques clés.

Notre projet vise ainsi à identifier les thématiques récurrentes dans les avis clients et à prédire la satisfaction à partir de ces informations. Nous avons structuré notre travail en plusieurs étapes : la collecte des avis, le nettoyage et la transformation des données, l'analyse exploratoire, puis l'application de techniques avancées comme le clustering et la modélisation des thématiques.

Les résultats obtenus peuvent être directement utilisés par les banques en ligne pour adapter leurs services et améliorer l'expérience utilisateur. De plus, la méthodologie développée est transposable à d'autres secteurs, offrant ainsi un cadre d'analyse robuste pour toute entreprise souhaitant exploiter les avis clients à grande échelle.

Ce rapport constitue un recueil descriptif du processus de projet. Il amène du contexte pour notre code concaténé en Python et décrit l'enchaînement de nos réflexions face aux challenges rencontrés.

Collecte et Exploration des Données

I. Web Scraping avec BeautifulSoup

De part nos appétences, nous avons choisi de dérouler ce projet autour de La satisfaction des client des Banques en Ligne en France, sans pour autant perdre en transposabilité de la méthodologie développée.

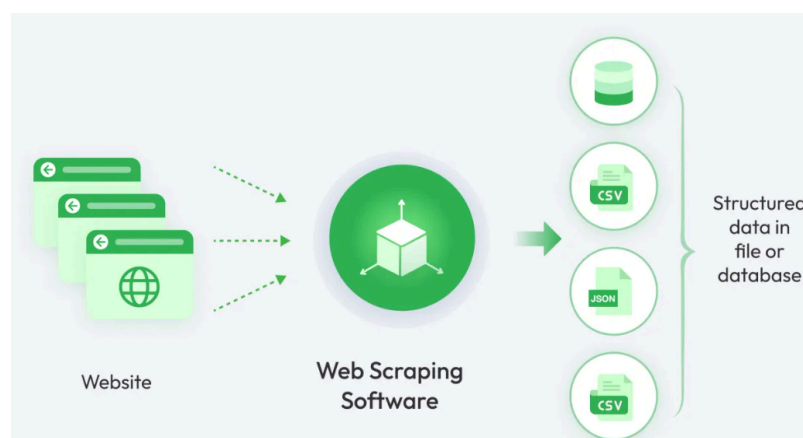
Nous avons choisi TrustPilot comme source de données car il y est précisé si les avis sont vérifiés ou non. Sa notoriété auprès des utilisateurs en fait une réelle mine d'avis pour les clients du BtoC.

La première étape consiste à récupérer des données. Pour cela, après avoir défini un site web cible pour notre secteur, nous avons testé trois bibliothèques différentes : Scrapy, Selenium et BeautifulSoup.

Nous avons comparé les 3 :

- **Scrapy** : Puissante mais plus adaptée aux grandes quantités de données nécessitant une gestion complexe.
- **Selenium** : Utile pour interagir avec des pages dynamiques, mais plus lourd et lent.
- **BeautifulSoup** : Solution retenue pour sa simplicité et son efficacité sur des pages statiques.

Nous avons retenu BeautifulSoup pour sa syntaxe simple et suffisamment robuste pour notre projet.



Process global du Web Scraping

Nous avons ainsi pu récupérer un fichier CSV par banque en ligne, qu'on a combiné en un dataframe unique. Nous avons choisi de ne conserver que les avis vérifiés. Ce data frame initial contient le nom de la banque, le titre des avis, l'identifiant du client ayant laissé l'avis, le champ de vérification, le rating, le commentaire et la date.

Lors de cette étape, nous avons rencontré un défi majeur lié au scraping de plusieurs pages successives. En effet, certaines plateformes limitent le nombre de requêtes envoyées dans un court laps de temps, ce qui peut entraîner des blocages ou des erreurs de récupération des données. Afin de contourner cette problématique, nous avons intégré une temporisation via la fonction `time.sleep(3)`, permettant d'espacer les requêtes et d'assurer le chargement complet des pages avant d'extraire les avis. Cette méthode nous a permis d'automatiser l'extraction des avis sur plusieurs banques en ligne et de sauvegarder ces données sous un format exploitable.

Fonctionnement du scraping :

Requête HTTP : Envoi d'une requête pour récupérer les pages contenant les avis.

1. **Parsing HTML** : Analyse et extraction des éléments pertinents (texte, notes, date).
2. **Stockage des données** : Sauvegarde sous format CSV avec des colonnes structurées (nom de l'entreprise, titre, ID, note, date, avis).

Nous avons fusionné les différentes bases de données collectées pour obtenir un jeu de données global et uniforme.

A la fin de cette étape nous avons récupéré un dataframe de 10 Colonnes et 188 759 lignes, contenant les avis des banques suivantes : BoursoBank, BforBank, Fortuneo, HelloBank, N26, Quanto, Revolut, MonaBank, OrangeBank, Oney, Nickel, MaFrenchBanque, Floabank, YounitedCredit et Cofidis

Notre DataFrame est prêt pour les étapes d'exploration et de modélisation.

	Company Name	titre	ID	verif	rating	avis	Date	month	day	year
0	boursobank.com	Carte piratée	GILLES DERELLE	Sur invitation	5	Je suis en vacances en Indonésie. Avis d'utili...	2024-10-09	10	9	2024
1	boursobank.com	Application bancaire fluide	Cyril P.	Sur invitation	5	Application bancaire fluide, rapide, ergonomiq...	2024-10-09	10	9	2024
2	boursobank.com	Parfait !	Bad Wolf	Non vérifié	5	Je ne peux que recommander BoursoBank. Bien qu...	2024-10-08	10	8	2024
3	boursobank.com	Connexion impossible	Jean-Paul Cauchin	Sur invitation	4	La connexion sur l'application mobile est parf...	2024-10-07	10	7	2024
4	boursobank.com	Je suis assez satisfaite des services...	Sophie R	Sur invitation	5	Je suis assez satisfaite des services de Bours...	2024-10-08	10	8	2024

II. Exploration et Pre-Processing

Après avoir vérifié la cohérence des types de variable et afficher la vue statistique du DataFrame, nous avons été amenés à le nettoyer.

Voici comment nous avons procédé :

Étape	Action réalisée	Résultat
1	Supprimer des doublons	9 433 doublons supprimés
2	Gérer des valeurs manquantes	Plus aucune valeur manquante (en fonction de la valeur). <i>Exemple : Si l'ID est manquant, nous définissons l'avis comme anonyme : "unknown"</i>
3	Transformer les dates	Dates correctement formatées
4	Nettoyer des textes	Suppression des caractères spéciaux, chiffres et accents

Lors de cette étape, nous avons constaté que l'index n'était pas actualisé. Nous avons rectifié cela grâce à la fonction `reset_index` en argument (`drop = True`).

En utilisant la méthode `map`, nous modifions le company name puis transformons le format de la date. Nous faisons également le choix de regrouper le titre et l'avis pour plus d'efficacité lors des analyses ultérieures.

	ID	verif	rating	Date	month	day	year	company	avis_global
0	GILLES DERELLE	Sur invitation	5	2024-10-09	10	9	2024	Boursorama	Carte piratée Je suis en vacances en Indonésie...
1	Cyril P.	Sur invitation	5	2024-10-09	10	9	2024	Boursorama	Application bancaire fluide Application banca...
2	Bad Wolf	Non vérifié	5	2024-10-08	10	8	2024	Boursorama	Parfait ! Je ne peux que recommander BoursoBan...
3	Jean-Paul Cauchin	Sur invitation	4	2024-10-07	10	7	2024	Boursorama	Connexion impossible La connexion sur l'applic...
4	Sophie R.	Sur invitation	5	2024-10-08	10	8	2024	Boursorama	Je suis assez satisfaite des services... Je suis...

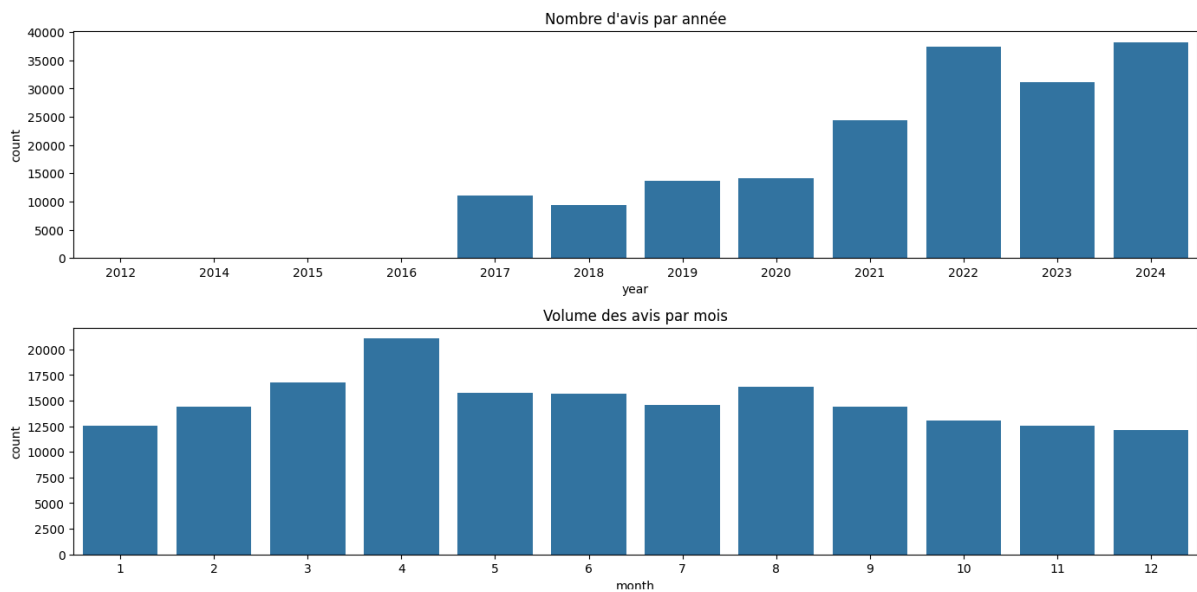
Par anticipation, nous clôturons cette étape par la création de variable de comptage pour préparer l'EDA

III - Présentation des visualisation retenues pour évaluer la pertinence des modèles utilisés plus tard

Les bibliothèques de visualisation des données utilisés dans ce document sont :

1. Matplotlib

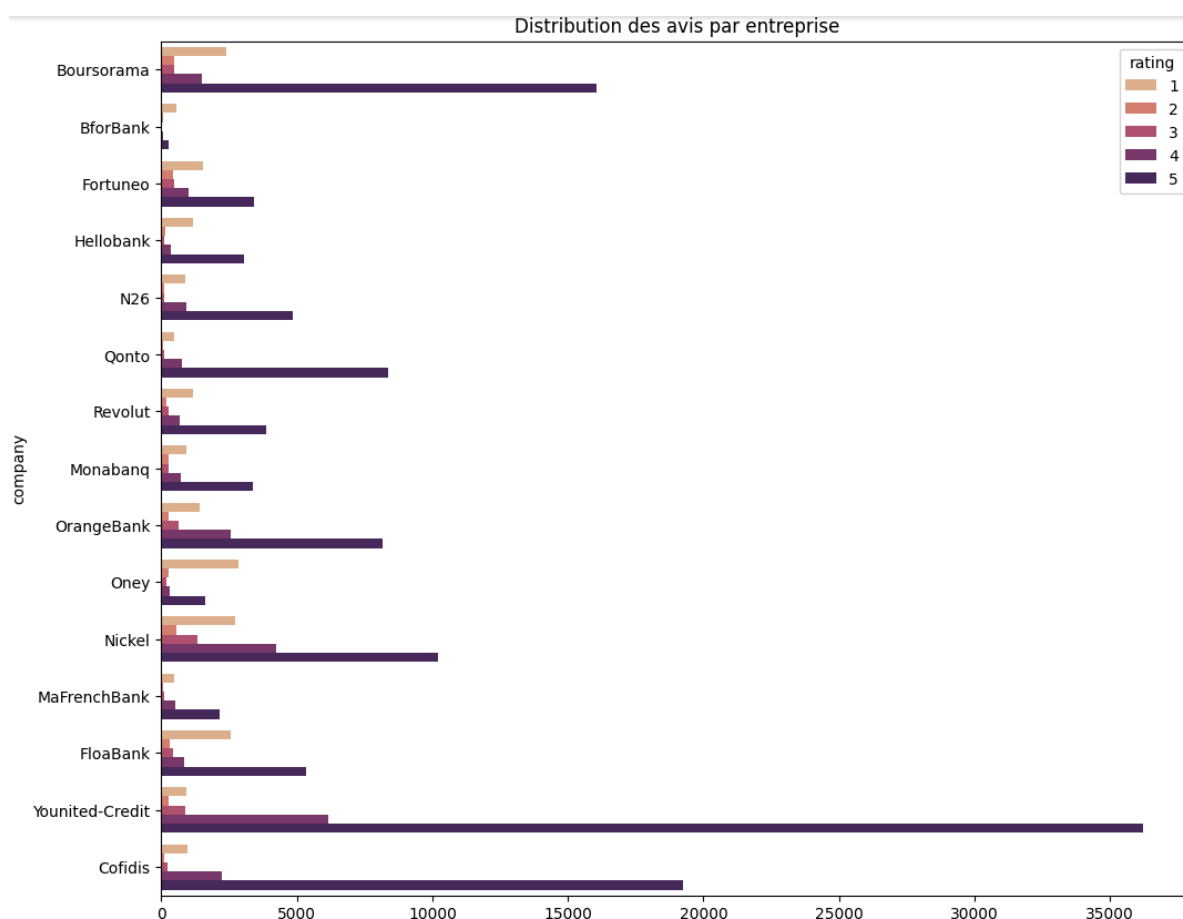
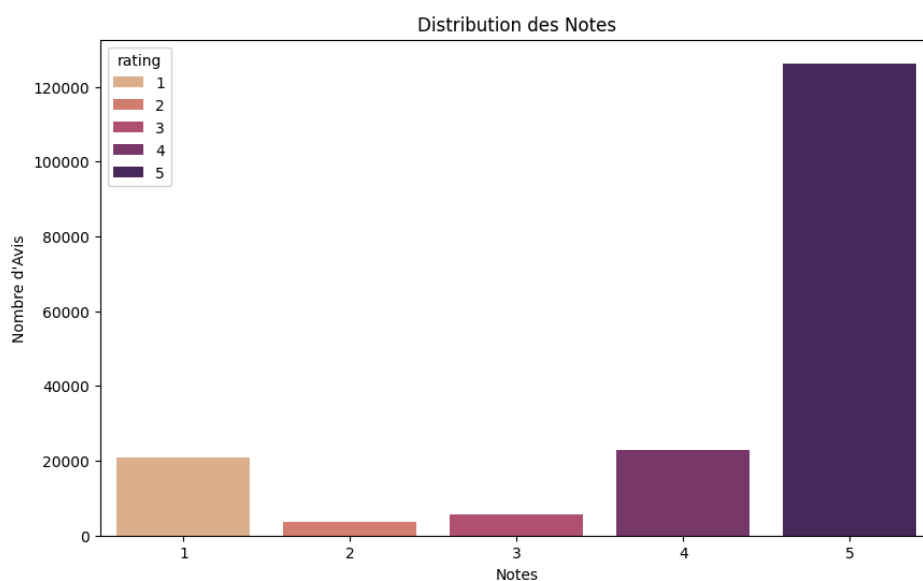
Répartition du nombre d'avis par année, pour montrer l'évolution du nombre d'avis collectés au fil des années et **Répartition du volume d'avis par mois** pour analyser la saisonnalité des avis



Le premier graphique montre une augmentation du volume d'avis chaque année à partir de 2016. Les années 2022 et 2024 sont celles où le nombre d'avis enregistré est le plus élevé. En 2023, nous remarquons une baisse significative du nombre d'avis (vs 2022). Cette baisse peut être expliquée par le retour à la normalité Post-Covid (retour des clients en banques traditionnelles). De plus, en 2023, TrustPilot a changé d'algorithme et de politique de modération. Plusieurs avis ont donc été supprimés par le site.

Le deuxième graphique, qui présente la répartition mensuelle des avis, montre qu'une moyenne d'environ 150 000 avis sont déposés chaque mois, à l'exception du mois d'avril qui est la meilleure période en termes de nombre d'avis déposés. Ceci s'explique par le renforcement de la publicité durant ce mois, par les banques en général, suite à la révision des taux et tarifs bancaires.

Distribution des notes attribuées pour montrer la fréquence de chaque note (1 à 5).



Ce graphique confirme que 5 est la note la plus fréquente dans la plupart des entreprises. Cependant, ce n'est pas la règle pour la note 1. Nous aimerions donc analyser la proportion d'évaluations par entreprise afin de pouvoir les comparer.

Distribution de la longueur des avis en fonction des notes (histogrammes empilés)

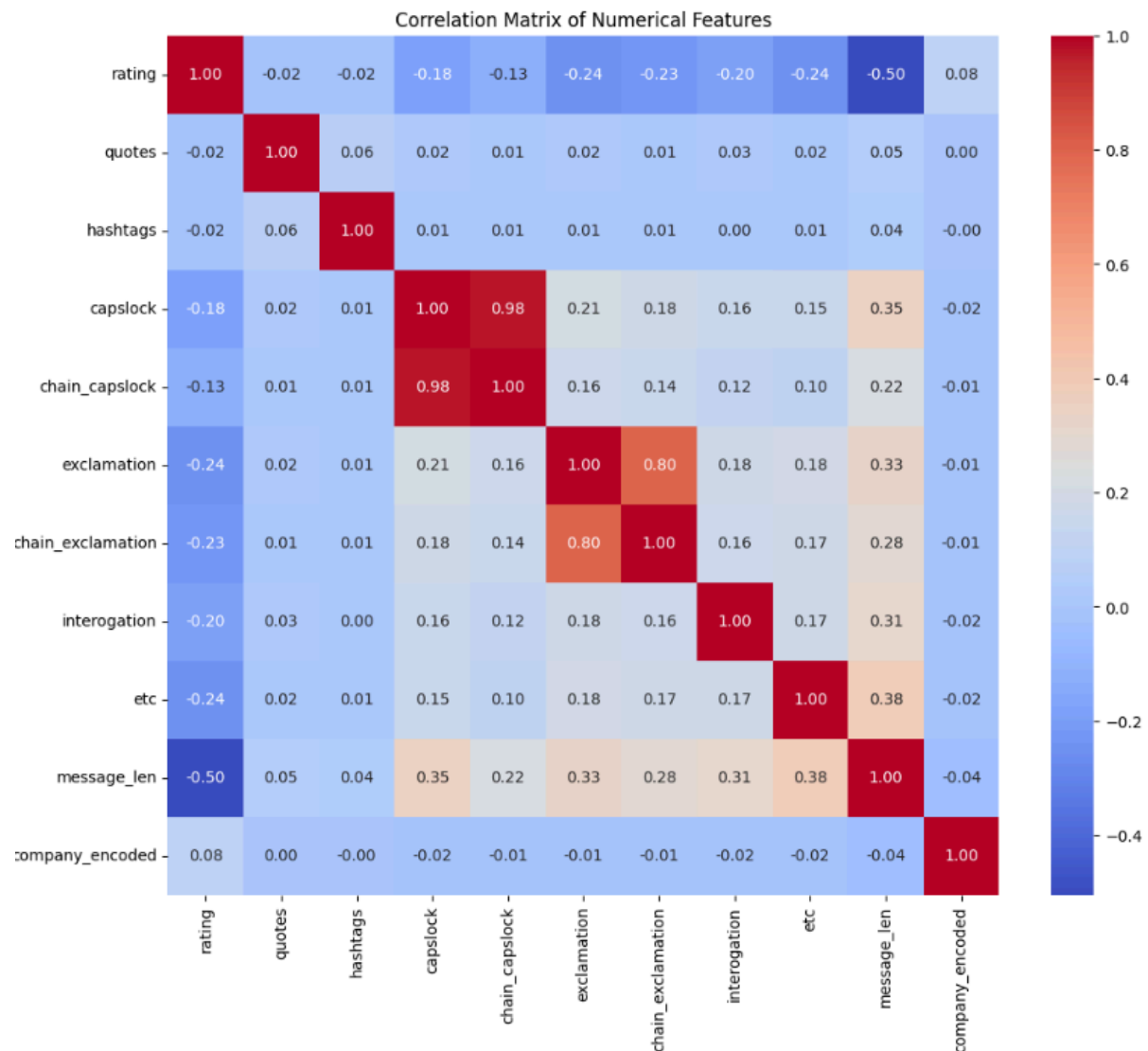
Ces histogrammes montrent la distribution de la longueur des avis pour chaque note.

Histogrammes de la longueur des avis par note (affichés en sous-graphiques)

Chaque sous-graphique représente la distribution de la longueur des avis pour une note spécifique.

Ces histogrammes sont utilisés pour explorer la distribution des avis, identifier les tendances et analyser les comportements des utilisateurs en fonction des notes attribuées. Ceci dit, dans notre cas, nous n'obtenons pas de corrélation visible entre ces différents paramètres par simple observations.

Matrice de Corrélation

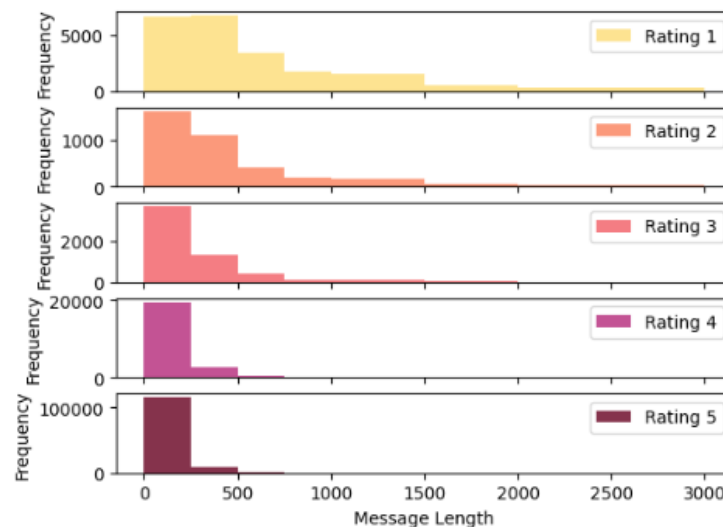


Nous constatons plusieurs corrélations positives et négatives, en particulier entre la note et la longueur de message -0.50. Nous décidons donc à présent suite à la visualisation du heatmap, en connaître plus sur la longueur des avis (nombre de caractères) pour chaque catégorie de note (1 à 5) à l'aide d'un histogramme.

2. **Seaborn** (création de graphiques statistiques comme countplot, FacetGrid, et barplots, pour la répartition des avis par année et par entreprise.)

Type de graphique	Fonction Seaborn utilisée	Objectif
Histogrammes	sns.histplot()	Visualiser la distribution des longueurs d'avis
Graphique en barres	sns.countplot()	Comparer le volume des avis par année, mois, entreprise Avec hue, les notes sont regroupées en positif (3, 4, 5) et négatif (1, 2). Nous avons joué avec cette répartition avant de trouver un résultat satisfaisant.
Graphique empilé	.plot(kind='bar', stacked=True, cmap='flare')	Permet de comparer la proportion des notes 1 à 5 par entreprise et le nombre total d'avis.
Graphique par catégorie	sns.FacetGrid()	Chaque sous-graphique représente la distribution de la longueur des avis en fonction de la note (1 à 5).

Pour notre besoin, nous retenons la vue suivante :



Nous constatons que la longueur des commentaires des clients satisfaits ayant obtenu la note 5 est très courte par rapport à celle des clients insatisfaits. Ce qui confirme la corrélation négative vu précédemment, nous pouvons envisager d'utiliser cette variable pour améliorer notre modèle si besoin.

3. WordCloud

WordCloud a été utilisé à plusieurs étapes pour explorer le contenu des avis et identifier les tendances générales et spécifiques aux entreprises

3.1. Première visualisation du nuage de mots (sans nettoyage du texte)

110

[illegible]

1. **Introduction**

- Autres mots génériques : plus, bien, encore, toujours

Nous utilisons nltk qui charge une liste de plus de 150 mots courants.

A cette étape, nous personnalisons ces stopwords avec des mots propres au secteur bancaire. Des mots comme "boursorama", "oney", "nickel" sont trop fréquents dans le dataset et doivent être supprimés car ils sont déjà contenus dans le nom de la banque.

Avis brut (avant nettoyage)	Avis nettoyé (après suppression des stopwords)
"Service client inefficace et long"	"inefficace long"
"Banque très bien, carte reçue rapidement"	"reçue rapidement"
"Impossible de contacter Oney pour un remboursement"	"impossible contacter remboursement"

Nous obtenons un texte allégé et plus pertinent pour l'analyse

Supprimer les accents, chiffres, caractères spéciaux et stopwords.

Effets du nettoyage :

Suppression des caractères spéciaux

Uniformisation des accents

Texte plus propre pour l'analyse

3.3 Suppression des stopwords pour affiner l'analyse

Supprimer les **mots inutiles** qui n'apportent pas de valeur analytique.

Effets du nettoyage :

Suppression des mots courants inutiles ("bien", "merci", "avoir")

Concentration sur les termes les plus significatifs

Conclusion Partielle :

L'analyse temporelle des avis a révélé plusieurs tendances intéressantes :

- Une **hausse constante** du nombre d'avis depuis 2016, reflétant l'adoption croissante des banques en ligne.
- Un **pic en 2022**, suivi d'une baisse en 2023, probablement due à des modifications dans les politiques de modération de TrustPilot.
- Une forte **saisonnalité**, avec un nombre d'avis significativement plus élevé en avril, période souvent marquée par des campagnes publicitaires bancaires.

Nous avons analysé la répartition des notes attribuées par les clients :

- La **note 5** est la plus fréquente, traduisant une majorité d'avis positifs.
- Contrairement aux attentes, la **note 1** n'est pas aussi dominante, ce qui suggère que la plupart des utilisateurs satisfaits laissent des avis, tandis que les mécontents s'expriment moins fréquemment.
- Nous avons identifié une **corrélation négative (-0.50) entre la longueur des avis et la note attribuée** : les avis négatifs sont souvent plus détaillés.

L'examen des avis via un **nuage de mots** a permis d'identifier des termes récurrents, mettant en avant des points forts (**simplicité, rapidité**) et des irritants (**service client, frais**)

Important : Cette section apparaît ici pour la cohérence du rapport. En réalité, nous avons décidé de débiter la modélisation avant le nettoyage complet du texte pour ne pas perdre en information. Le nettoyage et la visualisation des données nettoyées a été fait au milieu de l'étape de modélisation pour améliorer les modèles.

Ce processus s'est avéré complexe en raison des particularités de la langue française. Contrairement à l'anglais, le français comporte des accents, des conjugaisons variées et un vocabulaire riche qui rendent le nettoyage plus difficile. De plus, l'élimination des stop words doit être effectuée avec précaution afin de ne pas supprimer des mots qui pourraient être porteurs de sens. Par ailleurs, les avis clients incluent souvent des fautes d'orthographe ou un langage informel, ce qui complique leur traitement. Pour pallier ces difficultés, nous avons testé plusieurs listes de stop words et procédé à une vérification manuelle des données afin d'optimiser la qualité du texte nettoyé.

3.4. Deuxième WordCloud (après nettoyage du texte)

Générer un WordCloud amélioré avec un texte nettoyé.(Positifs 4 et 5 et Négatifs 1 à 3)



Résultat :

Apparition de mots-clés plus pertinents liés à la satisfaction client

Suppression des mots parasites

Visualisation plus claire des tendances

3.5. WordCloud par entreprise

Générer un nuage de mots spécifique à chaque entreprise.

Résultat :

- **Comparaison** des thèmes dominants entre entreprises.
- **Identification** des problèmes récurrents dans certaines banques.
- Meilleure **segmentation** des avis clients.

Modélisation et Prédiction

I- Vectorisation des Avis

TF-IDF et CountVectorizer

Nous avons converti les textes en représentations numériques grâce à deux méthodes essentielles :

- **Bag of Words (BoW)** : Création d'une matrice comptabilisant la fréquence des mots.
- **TF-IDF (Term Frequency-Inverse Document Frequency)** : Attribution d'un poids aux mots en fonction de leur importance relative dans le corpus.

Une fois les données textuelles prétraitées, il était nécessaire de les convertir en représentations numériques exploitables par les modèles d'analyse. Nous avons utilisé deux méthodes principales : Bag of Words (BoW), qui crée une matrice de fréquence des mots, et TF-IDF (Term Frequency-Inverse Document Frequency), qui pondère l'importance des termes en fonction de leur présence dans l'ensemble du corpus. Ces transformations permettent d'obtenir des vecteurs numériques décrivant chaque avis client.

Étapes de la vectorisation :

1. **Tokenisation des avis** : Transformation en unités de texte exploitables.
2. **Application de Count Vectorizer** : Création d'une matrice de comptage des mots.
3. **Utilisation de TF-IDF** : Pondération des mots en fonction de leur fréquence et de leur importance dans le corpus global.

Cependant, ces méthodes présentent des limitations. BoW considère tous les mots avec la même importance, ce qui peut inclure des termes non pertinents. TF-IDF corrige partiellement ce problème en attribuant un poids plus élevé aux mots rares et significatifs, mais peut sous-estimer l'importance de certains termes clés. De plus, le choix des hyperparamètres, comme **min_df** (fréquence minimale d'un mot pour être pris en compte) et **max_df** (fréquence maximale), impacte la qualité des résultats, dans notre cas : pour gérer les mots très fréquents mais peu informatifs (ex: "banque", "client")

Un mauvais réglage peut soit éliminer des mots utiles, soit inclure du bruit. Nous avons donc réalisé plusieurs expérimentations pour identifier les paramètres les plus pertinents et assurer une interprétation optimale des données vectorisées.

Bag of Words (BoW) permet de capturer la fréquence des mots, mais ne prend pas en compte leur importance contextuelle. À l'inverse, TF-IDF (Term Frequency-Inverse Document Frequency) pondère les mots en fonction de leur pertinence dans l'ensemble du corpus, mais peut attribuer un poids trop faible à des termes significatifs si leur fréquence globale est élevée, ce n'est pas une méthode suffisante, même en manipulant les hyperparamètres.

Cette étape nous a permis de quantifier les termes les plus récurrents et de mieux comprendre les tendances dans les avis clients, sans que ce soit explicitement concluant et exhaustif.

II - Clustering et Modélisation des Thématiques

Après avoir réussi à convertir chaque avis en une liste de mots individuels, nous souhaitons identifier automatiquement des thématiques, puis en regroupant les avis similaires en clusters pour mieux comprendre les tendances générales.

Pour cela nous avons utilisé deux algorithmes de modélisation thématique : LDA (Latent Dirichlet Allocation) et plus tard BERTopic.

LDA (Latent Dirichlet Allocation)

LDA est une méthode de **modélisation thématique** qui permet d'identifier automatiquement les sujets principaux contenus dans un ensemble de documents. Elle repose sur un modèle probabiliste qui regroupe les mots en topics cohérents, facilitant ainsi l'interprétation des grandes tendances des avis.

C'est un algorithme **non supervisé** qui identifie des groupes de mots récurrents pour définir des thématiques.

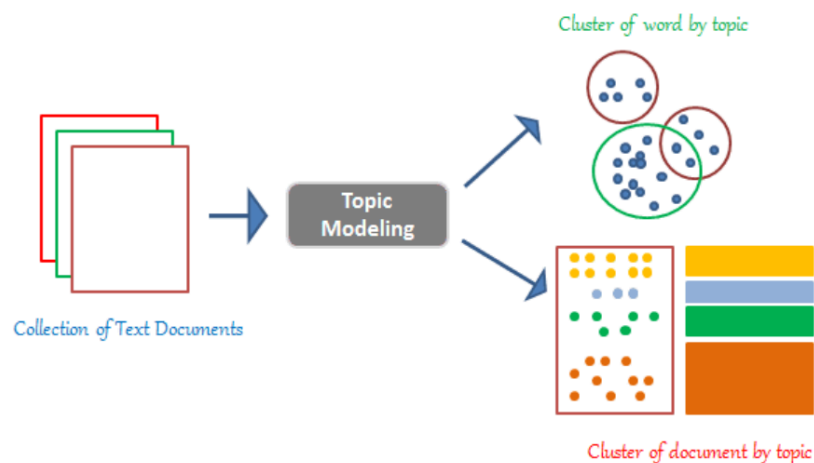


Schéma input/output du Topic Modeling

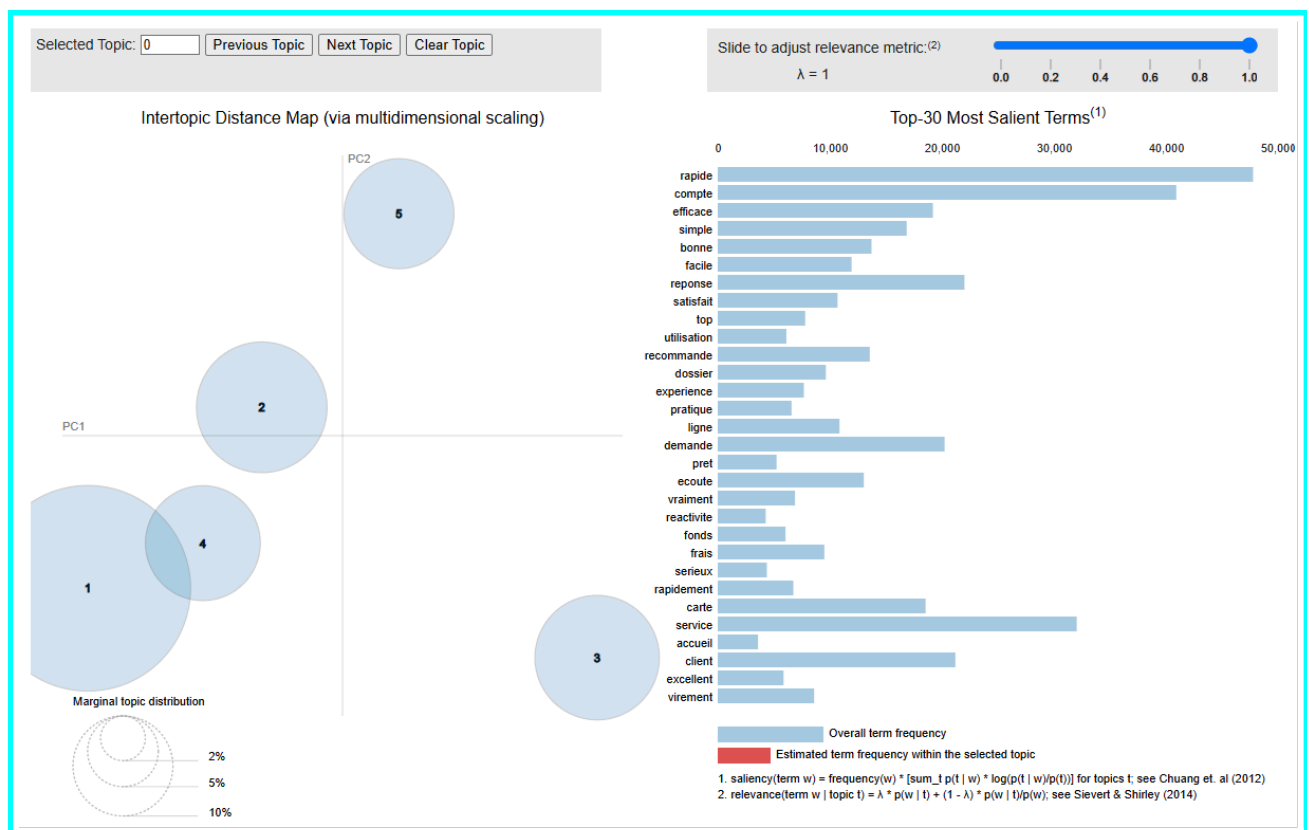
Fonctionnement de LDA et pré-requis:

1. **Transformation des avis en vecteurs numériques** via TF-IDF ou BoW.
2. **Choix du nombre optimal de topics** basé sur des métriques de cohérence.
3. **Attribution des mots aux thématiques** les plus pertinentes.
4. **Visualisation des résultats** pour interpréter les sujets récurrents.

Durant cette étape, nous avons regroupé nos avis en 15 puis en 5 clusters :

Cluster	Mots-clés associés	Interprétation
1	<i>rapide, efficace, service, écoute</i>	Satisfaction générale sur le service client
2	<i>demande, dossier, crédit, réponse</i>	Processus de prêt et validation des dossiers
3	<i>problème, connexion, application, bug</i>	Problèmes techniques avec l'application mobile
4	<i>taux, prêt, frais, virement</i>	Transactions bancaires et coûts associés
5	<i>facile, carte, paiement, rapide</i>	Simplicité des paiements et cartes bancaires

Nous avons ensuite tenté de visualiser l'éloignement entre ces clusters, en d'autre terme la bonne définition des clusters avec pyLDAvis : une carte interactive, permettant de comprendre l'interaction entre les thèmes.



L'un des principaux défis rencontrés lors de cette analyse a été le choix du nombre optimal de topics. Une valeur trop faible regroupe des thèmes trop larges et hétérogènes, tandis qu'un nombre trop élevé entraîne une fragmentation excessive des sujets. De plus, l'interprétation des résultats est parfois complexe : certains thèmes peuvent se chevaucher,

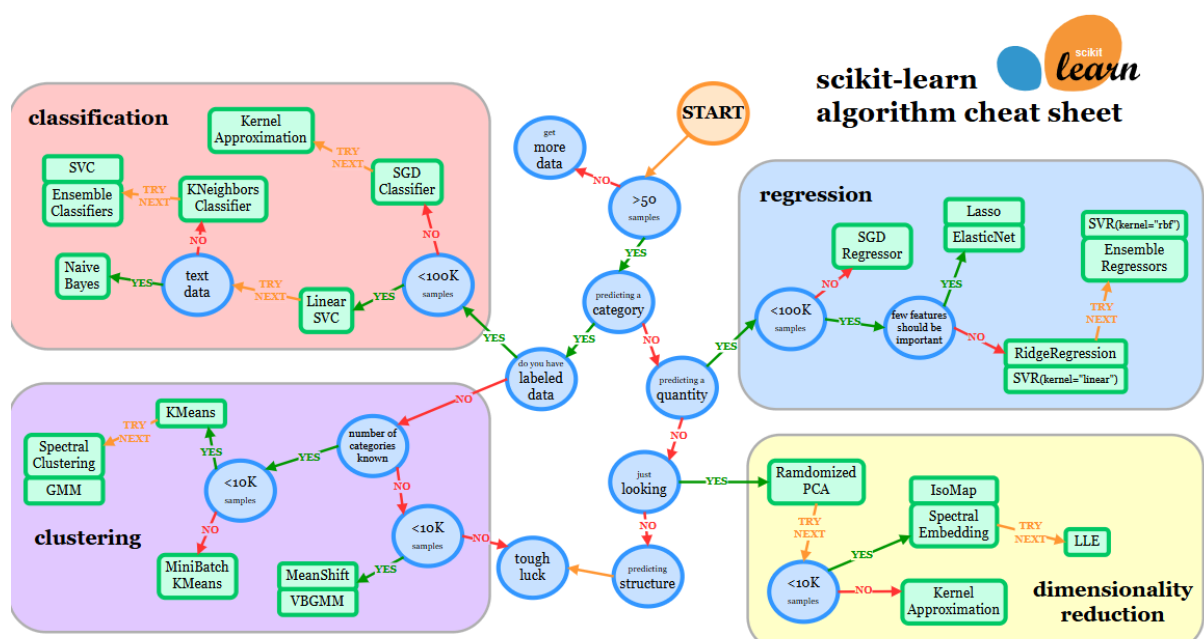
rendant leur identification difficile. Pour améliorer la robustesse de notre modèle, nous avons testé différentes métriques de cohérence thématique et comparé les performances avec différentes techniques de vectorisation (TF-IDF, BoW) en modifiant les paramètres.

Pendant l'exécution de ce projet, nous avons réitéré les méthodes. La LDA après l'étape Over Sampler.

III - Sélection de modèles

Nous tentons une première modélisation, sans retravailler les mots afin d'avoir une base de comparaison. Pour cela nous utilisons les algorithmes supervisés suivants décrits dans Scikit-Learn. Pour cela nous avons pris en compte plusieurs critères: (La taille, le type de donnée, le type prédiction, la structure et le format des données), ce qui nous a orienté vers ces trois catégories de modèles:

- a) Naive Bayes avec les méthodes de Bernoulli NB et Multinomial pour classer les avis (un classificateur qui suppose que l'effet d'une caractéristique particulière dans une classe est indépendant des autres caractéristiques).
- b) KNN - peut être considéré comme un système de vote, dans lequel le label de classe majoritaire détermine le label de classe d'un nouveau point de données parmi ses « k » (où k est un nombre entier) voisins les plus proches dans l'espace des caractéristiques.
- c) Logistic Regression - modèle statistique utilisé pour la classification et l'analyse prédictive pour estimer la probabilité qu'un événement se produise, en fonction d'un ensemble donné de variables indépendantes.



Nous créons des pipelines pour associer le CountVectorizer (pour convertir le text en caractéristiques numériques), le StandardScaler (pour mettre à l'échelle les données pour qu'elles aient une moyenne zero et une variance constante) et les classificateurs : BernoulliNB, MultinomialNB, KNN, et la régression Logistique pour la classification.

Nous calculons par la suite l'Accuracy, la précision, le rappel et affichons le rapport de classification pour évaluer les performances de chaque modèle.

Neighbors Classifier:					Bernoulli Naive Bayes:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
1	0.57	0.31	0.40	4122	1	0.62	0.72	0.66	4122
2	0.08	0.04	0.05	737	2	0.06	0.00	0.00	737
3	0.09	0.06	0.07	1088	3	0.16	0.05	0.08	1088
4	0.21	0.09	0.13	4528	4	0.29	0.24	0.26	4528
5	0.76	0.92	0.84	25391	5	0.85	0.90	0.87	25391
accuracy			0.70	35866	accuracy			0.75	35866
macro avg	0.34	0.28	0.30	35866	macro avg	0.39	0.38	0.38	35866
weighted avg	0.64	0.70	0.66	35866	weighted avg	0.71	0.75	0.73	35866

Multinomial Naive Bayes:					LogisticRegression:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
1	0.64	0.67	0.66	4122	1	0.72	0.67	0.70	4122
2	0.09	0.19	0.12	737	2	0.11	0.12	0.12	737
3	0.12	0.23	0.16	1088	3	0.14	0.15	0.15	1088
4	0.29	0.28	0.29	4528	4	0.32	0.19	0.24	4528
5	0.88	0.81	0.85	25391	5	0.85	0.91	0.88	25391
accuracy			0.70	35866	accuracy			0.75	35866
macro avg	0.40	0.44	0.41	35866	macro avg	0.43	0.41	0.42	35866
weighted avg	0.74	0.70	0.72	35866	weighted avg	0.73	0.75	0.74	35866

Nous avons obtenu les résultats

```
Précision:
Neighbors Classifier: 0.70
Bernoulli Naive Bayes: 0.75
Multinomial Naive Bayes: 0.70
LogisticRegression: 0.75
```

Nous pouvons constater que les scores extrêmes 1 et 5 sont mieux prédits que les scores intermédiaires. Les scores des 4 modèles de test sont moyens et peuvent être améliorés.

On observe que les modèles de Bernoulli et de Régression Logistique sont les plus performants (75%), contrairement aux modèles Multinomial et KNN.

Par ailleurs, nous constatons que l'utilisation de certains modèles tels que K-Nearest-Neighbors est très lente sur une grande base de données.

La **régression logistique** s'est révélée être le modèle le plus performant en raison de sa robustesse et de sa capacité à bien différencier les classes.

Nous avons essayé plusieurs approches, telles que la binarisation des notes ("target"), le nettoyage des mots et le rééquilibrage des données.

IV- Optimisation du modèle

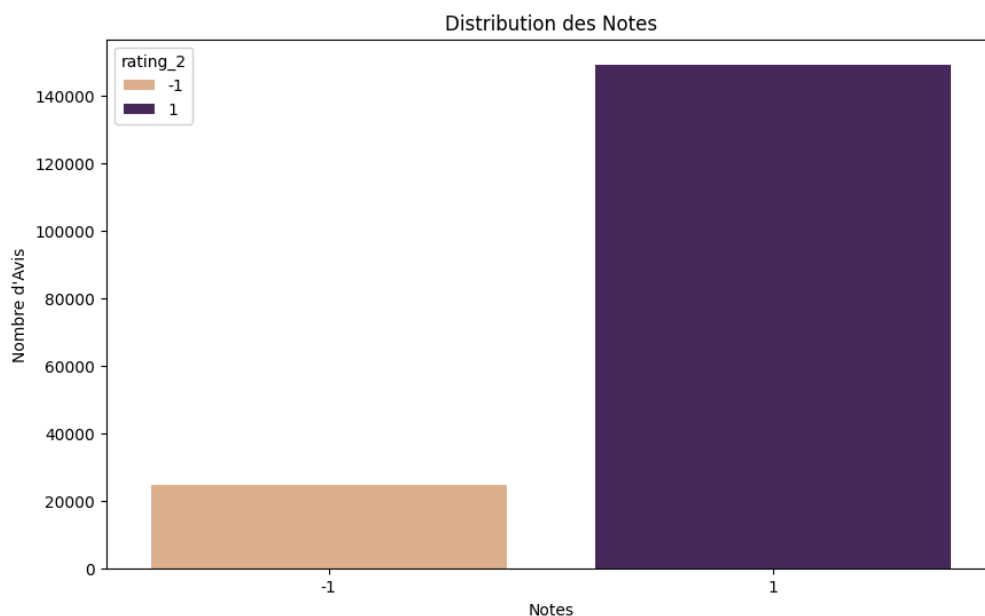
Afin d'améliorer la performance de notre modèle, nous avons mis en place plusieurs stratégies :

- **Binarisation des notes :**

Pour simplifier la modélisation, nous avons choisi de regrouper les avis en 2 catégories, grâce à la visualisation de la longueur des mots. Nous avons choisi de considérer les notes 4 & 5 comme positives et les notes 1 & 2 comme négatives, et de ne pas conserver la note 3 car elle est peu représentative en volume et elle est intermédiaire dans la grille de notation.

Nous avons testé le rééchantillonnage de nos données en utilisant des méthodes telles que RandomUnderSampler, RandomOverSampler de la bibliothèque imblearn, ainsi que le rééquilibrage manuel

- Regroupement des notes **1 et 2** en catégorie "Négatif"
- Regroupement des notes **4 et 5** en catégorie "Positif"



Nous constatons que le score négatif représente 14 % de notre ensemble de données avec cette binarisation. Le résultat est plus équilibré qu'en conservant la note 3 en positif ou en négatif.

Nous testons à nouveau les modèles précédents avec la binarisation de la cible avant d'envisager un rééquilibrage des données et nous obtenons :

--- BernoulliNB ---					--- KNN ---				
Classification Report:					Classification Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
-1	0.67	0.75	0.71	4800	-1	0.86	0.16	0.27	4800
1	0.96	0.94	0.95	29937	1	0.88	1.00	0.93	29937
accuracy			0.91	34737	accuracy			0.88	34737
macro avg	0.81	0.85	0.83	34737	macro avg	0.87	0.58	0.60	34737
weighted avg	0.92	0.91	0.92	34737	weighted avg	0.88	0.88	0.84	34737

--- MultinomialNB ---					--- LogisticRegression ---				
Classification Report:					Classification Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
-1	0.70	0.89	0.78	4800	-1	0.82	0.83	0.83	4800
1	0.98	0.94	0.96	29937	1	0.97	0.97	0.97	29937
accuracy			0.93	34737	accuracy			0.95	34737
macro avg	0.84	0.91	0.87	34737	macro avg	0.90	0.90	0.90	34737
weighted avg	0.94	0.93	0.93	34737	weighted avg	0.95	0.95	0.95	34737

En conclusion : La régression logistique est le modèle qui obtient le meilleur score 0.95. Cela va nous permettre grâce au bon résultat d'optimiser les paramètres de ce modèle et de rééquilibrer les données.

- **Techniques de rééquilibrage des données :**

Maintenant qu'un modèle émerge nous pouvons aborder le sujet des ensembles de données déséquilibrés. Il y a trois pistes de travail:

a) générer de nouveaux échantillons dans les classes sous-représentées (Over Sampling).

Méthodes populaires de suréchantillonnage des classes minoritaires :

(i) **RandomOverSampler** - suréchantillonnage en dupliquant certains des échantillons originaux de la classe minoritaire

(ii) la technique de suréchantillonnage synthétique des minorités (**SMOTE**) (**Synthetic Minority Over-sampling Technique**)- générer de nouveaux échantillons synthétiques différent sans distinction entre les échantillons faciles et difficiles à classer à l'aide de la règle des voisins les plus proches

(iii) la méthode d'échantillonnage synthétique adaptative (**ADASYN**) (**Adaptive Synthetic Sampling**)- génération d'échantillons à côté des échantillons originaux qui sont mal classés à l'aide d'un classificateur k-voisins les plus proches

b) diminuer les échantillons dans les classes surreprésentées (under Sampling) par exemple avec la méthode Tomeklinks (Il s'agit d'une modification des plus proches voisins condensés (CNN) et peut être utilisée pour trouver les échantillons de données souhaités de la classe majoritaire qui ont la distance euclidienne la plus faible avec les données de la classe minoritaire, puis pour les supprimer).

c) combiner les deux méthodes précédentes pour optimiser les résultats dpar exemple avec la méthode **SMOTE-TOMEK**.

Grâce à ces optimisations, la **régression logistique binaire a atteint une précision de 95%**, ne démontrant pas d'amélioration par rapport au modèle précédent. Notre méthode de standardisation n'est pas assez adaptée. Nous voulons tester le MinMax Scaler cependant il ne gère pas les matrices, ce qui nous oriente sur le MaxAbsScaler.

d) **MaxAbsScaler** : Normalisation des données pour éviter les biais liés à l'échelle des valeurs. Nous constatons que les différentes méthodes de rééquilibrages nous permettent d'améliorer le score de notre prédiction, nous tentons un rééquilibrage manuel également pour faire un choix.

```
Accuracy: 0.9014883265682125
Precision: 0.9078101448829555
Recall: 0.9858035207268597
```

	precision	recall	f1-score	support
-1	0.81	0.38	0.51	4800
1	0.91	0.99	0.95	29937
accuracy			0.90	34737
macro avg	0.86	0.68	0.73	34737
weighted avg	0.89	0.90	0.89	34737

Nous notons les bonnes performances, qui sont cependant moins bonnes que le RandomOverSampler. Nous continuons donc avec la méthode RandomOverSampler.

- **Recherche des hyperparamètres de la régression logistique avec Random Over Sampler**

Suite au calcul des hyperparamètres nous venons de trouver le meilleur résultat de notre modèle.

```
Classification Report:
```

	precision	recall	f1-score	support
-1	0.84	0.92	0.88	4800
1	0.99	0.97	0.98	29937
accuracy			0.97	34737
macro avg	0.92	0.95	0.93	34737
weighted avg	0.97	0.97	0.97	34737

Nous testons notre modèle en lui proposant un nouveau commentaire à analyser et nous constatons que le modèle prédit bien les sentiments que dégagent des commentaires positifs et négatifs

Conclusion Partielle : La partie modélisation nous à permi de calculer notre meilleur prédiction (97%), nous devons maintenant nous concentrer sur la détection des thématiques, pour cela nous avons travaillé avec les avis nettoyés.

V - Interprétation des résultats

Les performances du modèle montrent une bonne capacité à prédire la satisfaction client à partir des avis textuels. Toutefois, nous avons remarqué que :

- Les **notes extrêmes (1 et 5) sont mieux prédites** que les notes intermédiaires (2, 3, 4).
- Le **déséquilibre des classes impacte la capacité de prédiction des avis négatifs**, nécessitant un affinage du modèle.
- Une **analyse plus approfondie du contenu des avis** pourrait améliorer la précision du modèle en intégrant des techniques de NLP avancées.

VI- BERTopic

BERTopic est une méthode avancée de **modélisation des topics** basée sur des techniques de deep learning. Contrairement à LDA, qui repose sur des distributions probabilistes, BERTopic utilise des **embeddings BERT** pour représenter les documents dans un espace sémantique et applique un clustering HDBSCAN pour identifier les groupes de sujets les plus pertinents. Cette approche permet d'obtenir des thèmes plus précis et contextuellement cohérents.

Fonctionnement de BERTopic :

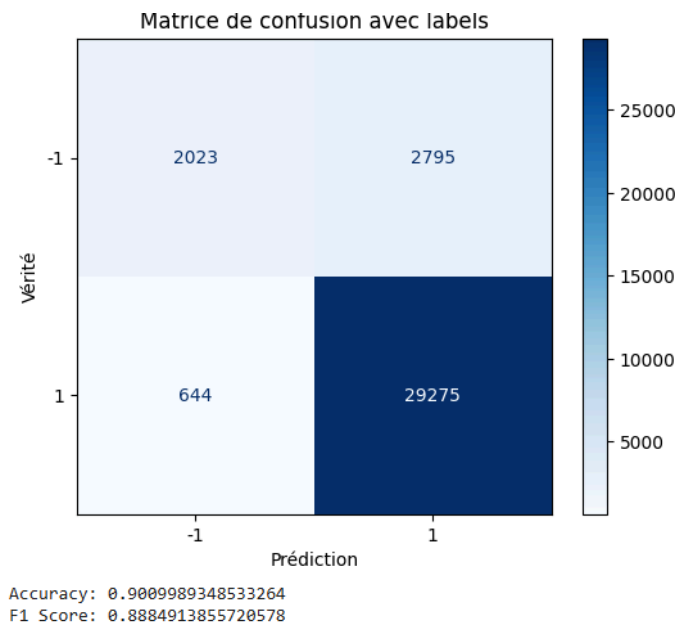
1. **Transformation des avis en embeddings BERT** pour une meilleure contextualisation des mots.
2. **Réduction de dimension** pour simplifier la structuration des données.
3. **Application du clustering avec HDBSCAN** pour regrouper les avis en thèmes homogènes.
4. **Extraction et analyse des termes clés** par thématique.

Cependant, cette méthode présente des contraintes importantes. L'utilisation d'embeddings BERT exige une puissance de calcul significative, ce qui allonge considérablement le temps d'exécution. De plus, le réglage des paramètres du clustering HDBSCAN est délicat : un paramètre mal ajusté peut générer des clusters trop larges ou, au contraire, trop spécifiques. Enfin, l'optimisation des modèles d'embeddings pour la langue française a nécessité des ajustements afin d'assurer une analyse correcte. Pour améliorer la performance de BERTopic, nous avons testé différentes méthodes de réduction de dimension et comparé les résultats avec ceux obtenus par LDA.

ANNEXE

ConfusionMatrixDisplay

Nous avons utilisé la matrice de confusion pour évaluer la performance des modèles de classification, principalement avec Bernoulli Naive Bayes et la régression logistique.



	Prédiction = Négatif (-1)	Prédiction = Positif (+1)
Réel = Négatif (-1)	3 580 (Vrai négatif) ✓	1 346 (Faux positif) ✗
Réel = Positif (+1)	2 077 (Faux négatif) ✗	28 863 (Vrai positif) ✓

Pour Bernoulli Naive Bayes, nous obtenons une bonne classification des avis positifs, sauf que le taux d'erreur est plus élevé sur les avis négatifs. On a un problème de déséquilibre des classes.

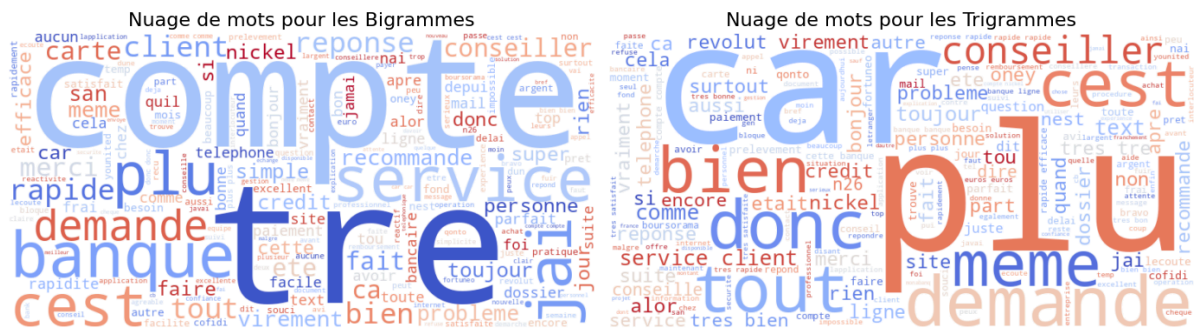
Pour corriger cela, nous avons testé différentes techniques comme le Random Under Sampler pour équilibrer les données et améliorer la matrice. SMOTE a amélioré la classification des avis négatifs.

Dans notre cas, Bernoulli Naive Bayes a donné les meilleurs résultats et SMOTE a amélioré la performance du modèle.

Les visualisations concernent principalement l'analyse exploratoire des données (EDA), l'analyse des sentiments à travers des nuages de mots et l'évaluation des performances des modèles de machine learning via des matrices de confusion.

Ces vues retenues ont permis de déterminer au fil du temps et des essais, la qualité de nos données, et puis leur pertinence.

Nuages de mots en Bigrammes et Trigrammes



Cette vue aurait pu nous permettre d'exploiter les adverbes et autres mots permettant d'accentuer un adjectif ou modifier son ton. Nous n'avons pas pu suivre cette piste par manque de temps.

Conclusion

Notre projet a permis d'identifier les principaux facteurs influençant la satisfaction client et de comparer différentes approches de modélisation thématique. L'analyse des avis clients offre aux banques en ligne un levier stratégique pour améliorer leurs services et fidéliser leur clientèle.

L'intégration de modèles avancés tels que BERT ouvre de nouvelles perspectives pour affiner encore l'interprétation des avis et prédire plus précisément les niveaux de satisfaction.

Pour notre exploration, la régression logistique binaire et l'hyperparamètre OverSampled nous ont permis d'obtenir le meilleur résultat possible, et la LDA nous a permis d'identifier ces thèmes.

Améliorations futures

- Tester des modèles NLP avancés tels que BERT ou GPT.
- Explorer de nouvelles méthodes de rééquilibrage des classes.
- Intégrer une analyse du ton et des émotions des avis.

Ce projet pose les bases d'un outil automatisé pour analyser la satisfaction client, avec des applications potentielles dans d'autres secteurs. Plus globalement, nous à travers ce projet, nous avons constaté l'importance de la recherche documentaire et de l'exploration des différentes bibliothèques existantes.

Références :

- <https://scikit-learn.org/stable/index.html>
- <https://docs.python.org/3.9/tutorial/>
- [Beautiful Soup Documentation](#)
- <https://larevueia.fr/latent-dirichlet-allocation-topic-modeling-en-python/>
- https://scikit-learn.org/stable/machine_learning_map.html